

PhoneBook — Step by Step Tutorial

PP2 · Programming Principles 2 | PostgreSQL + Python (psycopg2)

- i How to run the final program: `python phonebook.py` — make sure PostgreSQL is running and the database `phonebook_db` exists. All 7 steps below build up the complete `phonebook.py` file.

1 SETUP

Project setup & install psycopg2

Before writing any code, make sure `psycopg2` is installed and your project folder is ready.

A Install psycopg2

Open your terminal (PowerShell on Windows) and run:

Terminal

```
pip install psycopg2
```

B Create project folder & files

Create a folder `Practice7` with these files:

Structure

```
Practice7/  
phonebook.py <- main program  
contacts.csv <- sample data for import
```

C Create contacts.csv

Put this content into `contacts.csv`:

`contacts.csv`

```
first_name,phone  
Alice,+77001112233  
Bob,+77009998877  
Asel,+77711234567  
Daulet,+77759876543
```

- ! Always type file names and passwords in EN keyboard layout to avoid `UnicodeDecodeError`!

Connect to PostgreSQL

The very first block in `phonebook.py` — import libraries and create a connection to the database.

A Write the connection code

Add this to the top of `phonebook.py`:

`phonebook.py`

```
import psycopg2
import csv
import os

conn = psycopg2.connect(
    dbname="phonebook_db",
    user="Nurdaulet",
    password="Nur",
    host="localhost",
    port="5432"
)
cur = conn.cursor()
```

B Test the connection

Run the file — if no error appears, connection is working:

Terminal

```
python phonebook.py
```

- i `conn` = the connection to the DB. `cur` = cursor to execute SQL queries. Always call `conn.commit()` after INSERT / UPDATE / DELETE.

Design the PhoneBook table

Create the phonebook table with id, first_name, and phone columns. Safe to run multiple times.

A Write the create_table() function

phonebook.py

```
def create_table():
    cur.execute("""
    CREATE TABLE IF NOT EXISTS phonebook (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) NOT NULL UNIQUE
    );
    """)
    conn.commit()
    print("[OK] Table 'phonebook' is ready.")
```

B Verify the table was created

Run this inside the psql shell to check:

psql shell

```
\d phonebook
```

- O** id: auto-increment integer, primary key. first_name: max 100 chars, cannot be empty. phone: must be UNIQUE
- K** — no duplicates allowed.

Insert data from CSV file

Read contacts.csv and insert each row into the phonebook table. Duplicates are skipped automatically.

A Write insert_from_csv()

phonebook.py

```
def insert_from_csv(filepath: str):
    if not os.path.exists(filepath):
        print(f"[ERROR] File not found: {filepath}")
        return
    inserted = 0
    skipped = 0
    with open(filepath, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            name = row["first_name"].strip()
            phone = row["phone"].strip()
            try:
                cur.execute(
                    "INSERT INTO phonebook (first_name, phone)"
                    " VALUES (%s, %s)"
                    " ON CONFLICT (phone) DO NOTHING;",
                    (name, phone)
                )
                inserted += 1
            except Exception:
                skipped += 1
            conn.commit()
    print(f"[OK] {inserted} inserted, {skipped} skipped.")
```

B How to call it

phonebook.py

```
insert_from_csv("contacts.csv")
```

- i ON CONFLICT (phone) DO NOTHING — if the phone exists, skip that row. Always use %s placeholders, never f-strings in SQL (SQL injection risk).

Insert contact from console

Ask the user to type a name and phone number, then save it to the database.

A Write `insert_from_console()`

phonebook.py

```
def insert_from_console():
    print("\n-- Add new contact --")
    name = input(" First name : ").strip()
    phone = input(" Phone : ").strip()
    if not name or not phone:
        print("[ERROR] Name and phone cannot be empty.")
        return
    try:
        cur.execute(
            "INSERT INTO phonebook (first_name, phone)"
            " VALUES (%s, %s);",
            (name, phone)
        )
        conn.commit()
        print(f"[OK] Added: {name} -- {phone}")
    except psycopg2.errors.UniqueViolation:
        conn.rollback()
        print(f"[ERROR] Phone '{phone}' already exists.")
```

! Always call `conn.rollback()` in the `except` block after a failed `INSERT`. Without it, `psycopg2` refuses all further queries in the same transaction.

Update a contact's name or phone

Find a contact by name or phone, then let the user choose what to update.

A

Write `update_contact()`

phonebook.py

```
def update_contact():
    search = input(" Find by name or phone: ").strip()
    cur.execute(
        "SELECT id, first_name, phone FROM phonebook "
        "WHERE first_name ILIKE %s OR phone = %s;",
        (search, search)
    )
    rows = cur.fetchall()
    if not rows:
        print("[NOT FOUND]"); return
    for r in rows:
        print(f" id={r[0]} {r[1]} {r[2]}")
    contact_id = input(" Enter id to update: ").strip()
    print(" 1 - name 2 - phone")
    choice = input(" Choice: ").strip()
    if choice == "1":
        new_val = input(" New name: ").strip()
        cur.execute("UPDATE phonebook SET first_name=%s WHERE id=%s;",
                    (new_val, contact_id))
    elif choice == "2":
        new_val = input(" New phone: ").strip()
        cur.execute("UPDATE phonebook SET phone=%s WHERE id=%s;",
                    (new_val, contact_id))
    conn.commit()
    print("[OK] Updated.")
```

i

ILIKE is case-insensitive LIKE — searching 'alice' also finds 'Alice'. Show all matches first, then let the user pick an id to be precise.

Query & Delete contacts

Two functions: one to search with filters, one to delete by name or phone.

A Write `query_contacts()`

phonebook.py

```
def query_contacts():
    print(" 1-all 2-by name 3-by phone prefix")
    choice = input(" Choice: ").strip()
    if choice == "1":
        cur.execute("SELECT id, first_name, phone"
                    " FROM phonebook ORDER BY first_name;")
    elif choice == "2":
        name = input(" Name (partial OK): ").strip()
        cur.execute(
            "SELECT id, first_name, phone FROM phonebook"
            " WHERE first_name ILIKE %s ORDER BY first_name;",
            (f"%{name}%",)
        )
    elif choice == "3":
        prefix = input(" Phone prefix (e.g. +7700): ").strip()
        cur.execute(
            "SELECT id, first_name, phone FROM phonebook"
            " WHERE phone LIKE %s ORDER BY phone;",
            (f"{prefix}%",)
        )
    rows = cur.fetchall()
    for r in rows:
        print(f" {r[0]:<4} {r[1]:<20} {r[2]}")
```

B Write delete_contact()

phonebook.py

```
def delete_contact():
    print(" 1-by name 2-by phone")
    choice = input(" Choice: ").strip()
    if choice == "1":
        name = input(" Name: ").strip()
        cur.execute(
            "DELETE FROM phonebook WHERE first_name ILIKE %s"
            " RETURNING first_name, phone;", (name,)
        )
    elif choice == "2":
        phone = input(" Phone: ").strip()
        cur.execute(
            "DELETE FROM phonebook WHERE phone = %s"
            " RETURNING first_name, phone;", (phone,)
        )
    deleted = cur.fetchall()
    conn.commit()
    if deleted:
        for d in deleted:
            print(f"[OK] Deleted: {d[0]} -- {d[1]}")
    else:
        print("[NOT FOUND]")
```

- O** RETURNING lets you confirm which rows were deleted. Using %{name}% in ILIKE means 'contains' — 'Al'
- K** matches both 'Alice' and 'Asel'.